اینجا، سه رویکرد پایه‌ای را بحث می‌کنیم. اولی، یک رابط خط فرمان یا **مفسر فرمان**[1] فراهم می‌کند که کاربران می‌توانند مستقیماً فرمان‌هایی را وارد کنند تا توسط سیستم‌عامل انجام شوند. دو مورد دیگر به کاربران اجازه می‌دهد تا از طریق یک رابط کاربر گرافیکی یا GUI با سیستم‌عامل ارتباط برقرار کنند.

۲-۲-۱ مفسرهای فرمان

اکثر سیستم‌عامل‌ها اعم از لینوکس، یونیکس، و ویندوز، مفسر فرمان را به صورت برنامه‌ی خاصی می‌بینند که در زمان آغاز یک پردازه، یا زمانی که کاربر برای اولین‌بار به سیستم وارد می‌شود (در سیستم‌های محاوره‌ای) به اجرا در می‌آید. در سیستم‌هایی که مفسرهای فرمان متعددی برای انتخاب دارند، مفسرها به **پوسته‌ها**[2] معروفند. برای مثال، در سیستم‌های یونیکس و لینوکس، کاربر می‌تواند از میان چندین پوسته‌ی متفاوت، اعم از *C shell*، *Bourne-Again shell*، *Korn shell* و سایرین انتخاب داشته باشد. پوسته‌های بازاری و پوسته‌های کاربرنویس رایگان نیز وجود دارند. اکثر پوسته‌ها کارکرد مشابهی فراهم می‌کنند، و این که، یک کاربر کدام پوسته را برای استفاده انتخاب می‌کند عموماً به سلیقه‌ی فردی بستگی دارد. شکل ۲-۲، مفسر فرمان پوسته‌ی Bourne-Again (یا bash) را که در macOS استفاده می‌شود، نشان می‌دهد.

کارکرد اصلی مفسر فرمان، دریافت و اجرای فرمان بعدی است که کاربر مشخص می‌کند. بسیاری از فرمان‌هایی که در این سطح صادر می‌شوند جهت دستکاری فایل‌هاست که عبارتند از: ایجاد، حذف، فهرست‌گیری، چاپ، کپی، اجرا و از این دست. پوسته‌های مختلفی که در سیستم‌های یونیکس در دسترس هستند با این شیوه کار می‌کنند. این فرمان‌ها می‌توانند به دو روش کلی پیاده‌سازی شوند.

در یک رویکرد، خود مفسر فرمان، کد اجرایی هر یک از فرمان‌ها را داراست. برای مثال، صدور فرمان حذف یک فایل باعث می‌شود که مفسر فرمان به بخشی از کد خود پرش کرده و در آنجا با تنظیم پارامترهایی، فراخوانی سیستمی مناسبی را انجام بدهد. در اینجا، تعداد فرمان‌هایی که صادر می‌شوند اندازه‌ی مفسر فرمان را تعیین می‌کنند، چرا که که هر فرمان، کد پیاده‌ساز خاص خود را لازم دارد.

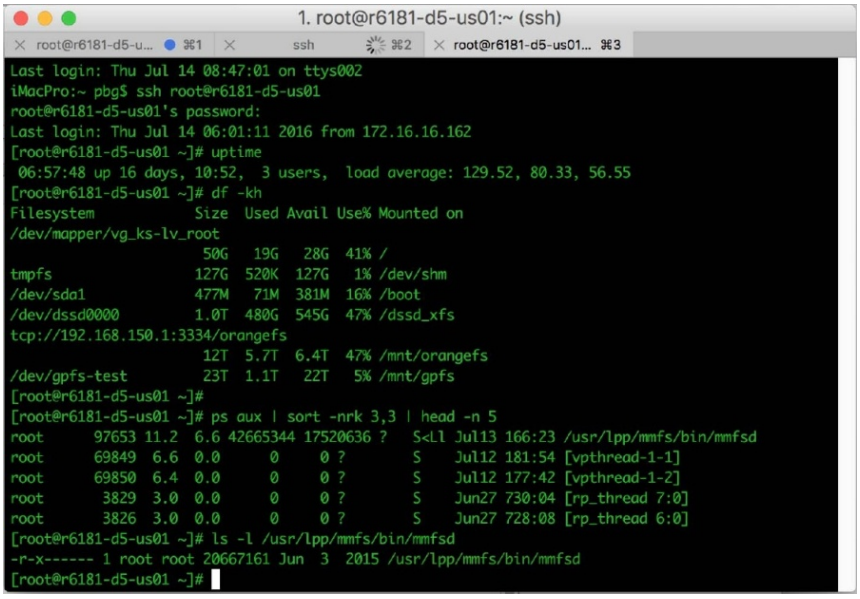
رویکرد دیگری هم هست که—در میان سیستم‌عامل‌ها توسط یونیکس استفاده می‌شود—اکثر فرمان‌ها را از طریق برنامه‌های سیستمی پیاده‌سازی می‌کند. در اینجا، مفسر فرمان هیچ‌گونه شناختی از فرمان‌ها نداشته و تنها از فرمان برای شناسایی فایلی که داخل حافظه بارگذاری و اجرا می‌شود، استفاده می‌کند. بنابراین، فرمان حذف یک فایل در یونیکس به صورت زیر:

```
rm file.txt
```

فایلی به نام rm را جستجو و در داخل حافظه بار نموده و با پارامتر file.txt اجرا می‌کند. منطق مرتبط با فرمان rm به طور کامل، با کد درون فایل rm تعریف می‌شود. با این شیوه، برنامه‌نویسان می‌توانند فرمان‌های جدیدی را به آسانی با ایجاد فایل‌های جدید با منطق برنامه‌ی

1. Command interpreter
2. Shells

مناسب اضافه کنند. برنامه‌ی مفسر فرمان هم که می‌تواند کوچک باشد، نیازی نیست که با افزودن فرمان‌های جدید تغییر یابد.



شکل ۲-۲ مفسر فرمان پوسته‌ی bash در macOS.

۲-۲-۲ رابط کاربر گرافیکی

راهبرد دوم ارتباط با سیستم‌عامل از طریق رابط کاربر گرافیکی کاربرپسند یا GUI است. در اینجا، کاربران به جای این که مجبور باشند فرمان‌ها را مستقیماً از طریق یک رابط خط فرمان وارد کنند، از سیستم پنجره و منوی مبتنی بر ماوس استفاده می‌کنند که با یک اشاره‌گر متحرک در میزکار مشخص می‌شود. کاربر، ماوس را حرکت می‌دهد تا اشاره‌گر آن، روی تصاویر، یا **شمایل**[1] صفحه (میزکار) قرار بگیرد که معرف برنامه‌ها، فایل‌ها، دایرکتوری‌ها و کارکردهای سیستم می‌باشند. بسته به محل اشاره‌گر ماوس، کلیک دکمه‌ی ماوس می‌تواند برنامه‌ای را احضار کند، یک فایل یا دایرکتوری—معروف به **پوشه**—را انتخاب نماید یا منویی را باز کند که فرمان‌هایی را داراست.

رابط‌های کاربر گرافیکی برای اولین بار در اوایل دهه‌ی ۱۹۷۰، در انجام بخشی از تحقیقات Xerox PARC ظهور کردند. اولین GUI خود را در کامپیوتر Xerox Alto در سال ۱۹۷۳ نشان داد. با این وجود، رابط‌های گرافیکی با پیدایش کامپیوترهای Apple Macintosh در دهه‌ی ۱۹۸۰ گسترش بیشتری پیدا کردند. رابط کاربر سیستم‌عامل مکینتاش نیز تغییرات گوناگونی پشت سر گذاشته است، مهمترین آن رابط Agua می‌باشد که با macOS عرضه شد. اولین نسخه‌ی ویندوز

1. Icons

مایکروسافت—نسخه‌ی 1.0— بر اساس افزودن یک رابط GUI به سیستم‌عامل MS-DOS بود. نسخه‌های متعددی از سیستم‌های ویندوز که بعدها از نسخه‌ی اولیه عرضه شدند برای داشتن ظاهری بهتر، مرتب تغییر چهره داده و پیشرفت‌های متعددی در کارکرد خود داشته‌اند.

به طور سنتی، سیستم‌های یونیکس تحت سلطه‌ی رابط‌های خط فرمان بودند. با این همه، با توسعه‌های چشم‌گیر در طراحی‌های GUI پروژه‌های **متن‌باز**، همچون محیط میزکار K[1] (یا KDE) و میزکار GNOME پروژه‌ی GNU، رابط‌های GUI گوناگونی در دسترس هستند. میزکارهای KDE و GNOME، هر دو در لینوکس و سیستم‌های مختلف یونیکس اجرا شده و تحت مالکیت متن‌باز در دسترس هستند، به این معنی که کد اصلی آنها به سادگی برای خواندن و تغییر با مجوزهای خاص، فراهم و در دسترس می‌باشد.

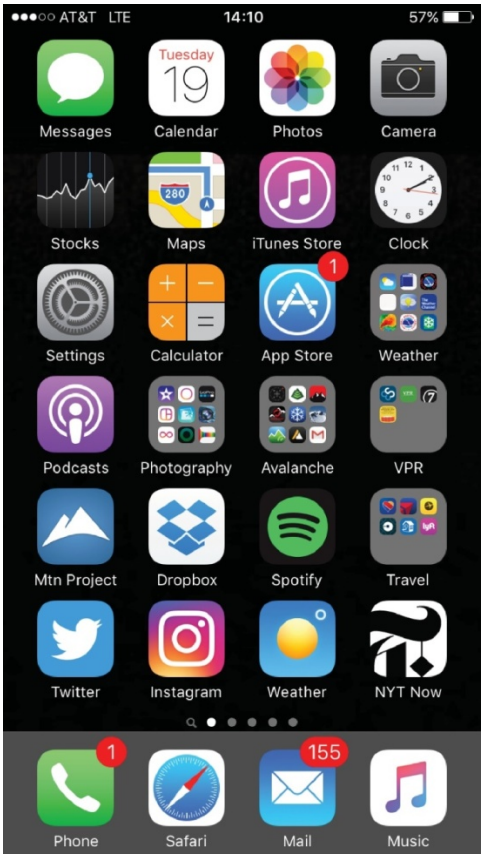
۲-۲-۳ رابط صفحه‌نمایش لمسی

چون یک رابط خط‌فرمان و یا سیستم ماوس و صفحه‌کلید برای اکثر سیستم‌های موبایل غیر عملی است، گوشی‌های هوشمند و کامپیوترهای تبلت دستی نوعاً از رابط نمایشگر لمسی استفاده می‌کنند. در اینجا، کاربران با **اشاراتی**[2] بر روی نمایشگر لمسی–برای مثال، فشردن و کشیدن انگشتان در عرض صفحه–ارتباط دارند. در حالی‌که گوشی‌های هوشمند پیشین، یک صفحه‌کلید فیزیکی کار می‌گذاشتند، اکثر گوشی‌های هوشمند فعلی، صفحه‌کلید را بر روی نمایشگر لمسی شبیه‌سازی می‌کنند. شکل ۲-۳، نمایشگر لمسی iPhone اپل را به‌تصویر می‌کشد. هر دوی iPad و iPhone از رابط صفحه‌نمایش لمسی **Springboard** استفاده می‌کنند.

۲-۲-۴ انتخاب رابط

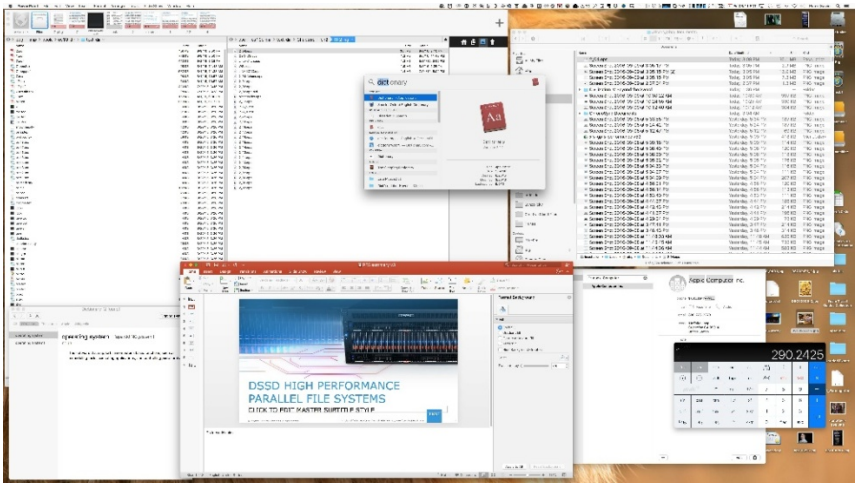
انتخاب کار با رابط خط فرمان یا GUI، اغلب به سلیقه‌ی کاربر بستگی دارد. **مدیران سیستم** که کامپیوترها را مدیریت می‌کنند و **power user**ها که دانش عمیقی از یک سیستم دارند به کرات از رابط خط فرمان استفاده می‌کنند. این کار برای آنها کارآمد است، چرا که دسترسی سریعی در انجام فعالیت‌های مطلوبشان فراهم می‌کند. در واقع، در برخی سیستم‌ها، تنها زیرمجموعه‌ای از کارکردها از طریق GUI قابل دسترس هستند و وظایفی که رواج کمتری دارند برای آنهایی که دانش خط فرمان دارند، کنار گذاشته می‌شوند. به علاوه، رابط‌های خط فرمان معمولاً کارهای تکراری را تسهیل می‌بخشند، چرا که ویژگی برنامه‌پذیری دارند. برای مثال، اگر یک کار مکرر، نیازمند یک مجموعه مراحل خط فرمان است، می‌توان آن مراحل را در یک فایل ذخیره کرده و درست مثل یک برنامه اجرا کرد. در اینجا، برنامه به کد اجرایی کامپایل نمی‌شود، بلکه توسط مفسر خط فرمان تفسیر می‌شود. این مدل از **اسکریپت‌های پوسته** در سیستم‌هایی همچون یونیکس و لینوکس که گرایش به خط فرمان دارند، بسیار رایج است.

1. K Desktop Environment
2. Gestures



شکل ۲-۳ نمایشگر لمسی iPhone.

در مقایسه، اکثر کاربران ویندوز از محیط GUI ویندوز لذت می‌برند و تقریباً هرگز از رابط پوسته استفاده نمی‌کنند. نسخه‌های اخیر سیستم‌عامل ویندوز، توأمان یک GUI استاندارد برای سیستم‌های رومیزی و لپ‌تاپ‌های قدیمی و یک صفحه‌نمایش لمسی برای تبلت‌ها فراهم می‌کنند. سیستم‌عامل‌های مکینتاش با تغییرات مختلفی که داشته‌اند زمینه‌ی مطالعه‌ی خوبی هم برای مقایسه فراهم می‌کنند. در گذشته، macOS فاقد رابط خط فرمان بود و همواره کاربران خود را ملزم می‌کرد که با استفاده از GUI با سیستم‌عامل ارتباط داشته باشند. با این همه، با عرضه‌ی نسخه‌ی macOS (که در قسمتی از خود از هسته‌ی یونیکس بهره می‌برد)، یک Agua GUI و نیز یک رابط خط فرمان فراهم کرده است. شکل ۲-۴، تصویر لحظه‌ای رابط کاربر گرافیکی macOS را نشان می‌دهد.



شکل ۲-۴ رابط کاربر گرافیکی macOS.

هرچند که برنامه‌هایی هستند که یک رابط خط‌فرمان برای سیستم‌های موبایل iOS و اندروید فراهم می‌کنند، آن‌ها به‌ندرت استفاده می‌شوند. در عوض، تقریباً تمامی کاربران سیستم‌های موبایل با استفاده از رابط صفحه‌نمایش لمسی با دستگاه‌های خودشان تعامل برقرار می‌کنند.

رابط کاربر می‌تواند از سیستمی به سیستم دیگر و حتی از کاربری به کاربر دیگر در میان یک سیستم فرق کند؛ با این وجود، نوعاً در اصل از ساختار واقعی سیستم برداشته می‌شود. لذا، طراحی یک رابط کاربری مفید و مشهود، یک کارکرد مستقیم سیستم‌عامل نیست. در این کتاب، ما بر روی مسئله‌های اساسی تدارک سرویس کافی برای برنامه‌های کاربر تمرکز می‌کنیم. از نقطه‌نظر سیستم‌عامل، ما بین برنامه‌های کاربر و برنامه‌های سیستم تفاوتی قایل نمی‌شویم.

۲-۳ فراخوانی‌های سیستمی

فراخوانی‌های سیستمی[1] رابطی برای سرویس‌های سیستم‌عامل فراهم می‌کنند. این فراخوانی‌ها عموماً به صورت روال‌هایی به زبان C و C++ هستند، هرچند ممکن است وظایف سطح پایین خاصی (برای مثال، وظایفی که دسترسی مستقیم سخت‌افزار را لازم دارند) به ناچار با استفاده از دستورات زبان اسمبلی هم نوشته شوند.

۲-۳-۱ مثال

پیش از این که به بحث چگونگی تدارک فراخوانی‌های سیستمی توسط سیستم‌عامل وارد شویم، اجازه بدهید تا نحوه‌ی استفاده از فراخوانی‌های سیستمی را با ذکر یک مثال روشن کنیم:

1. System calls